

GUIA TECNICA INTEGRAL JAVA + SPRING BOOT

De conceptos basicos a tecnicos, explicados con el codigo real
del proyecto **Ventas-Backend**

Preparacion entrevista **BBVA - Desarrollador Fullstack**

Dato	Valor
Stack real del proyecto	Spring Boot 4.0.5, Java 21, Maven, Spring Data JPA (Hibernate), Spring Security + JWT, PostgreSQL, Docker

Indice de contenidos

1. La JVM, JDK y el ciclo de vida de un programa Java
2. Fundamentos del lenguaje aplicados al proyecto
3. Programacion Orientada a Objetos (POO)
4. Maven y Spring Initializr
5. Spring Core: Inversion de Control (IoC) y DI
6. Spring Boot: auto-configuracion, starters y profiles
7. Spring MVC: construccion de APIs REST
8. Spring Data JPA: entidades, repositorios y JPQL
9. Transacciones y el problema N+1
10. Spring Security: JWT, filtros y autorizacion
11. Manejo global de excepciones
12. DTOs y Mappers
13. Testing: JUnit 5, Mockito, MockMvc

14. Temas avanzados de entrevista

15. Caso de estudio end-to-end: flujo de una venta

1. La JVM, el JDK y el ciclo de vida de un programa Java

1.1 JDK vs JRE vs JVM

- **JDK (Java Development Kit)**: kit de desarrollo. Incluye el compilador `javac`, el JRE, herramientas (jar, jlink) y librerías. Lo usa el que desarrolla.
- **JRE (Java Runtime Environment)**: solo el entorno para EJECUTAR programas compilados. No trae compilador.
- **JVM (Java Virtual Machine)**: la máquina virtual. Ejecuta el **bytecode** (.class), es la clave del famoso *"write once, run anywhere"*: el mismo .class corre en Windows, Linux o macOS porque cada plataforma tiene su propia JVM.

En el proyecto, `VentasApplication` es la clase con el método `main`. Al ejecutar `mvnw.cmd spring-boot:run` (o `java -jar app.jar` en Docker), la JVM arranca, carga el classpath y llama a `SpringApplication.run(...)`, que levanta el contenedor de Spring.

1.2 Compilacion y bytecode

El flujo es: **código fuente** (.java) -> **javac** -> **bytecode** (.class) -> **JVM** -> código máquina (JIT). Maven automatiza ese proceso; por eso el `target/` del proyecto contiene los .class compilados y el jar final.

En una entrevista, di que la JVM usa **Just-In-Time (JIT)**: interpreta el bytecode y detecta hot paths para compilarlos a código nativo en caliente, y que desde Java 9 existe **JAR modular** y que **GraalVM** permite native image (aunque aquí se usa JVM clásica).

1.3 Tipos de datos y wrappers

En el proyecto **nunca** usamos `double` para dinero: usamos `BigDecimal`. Es una pregunta clásica de entrevista:

- **Primitivos**: int, long, double, boolean, char, byte, short, float. Guardan el valor en un espacio fijo de memoria.
- **Wrappers**: Integer, Long, Double, Boolean... objetos que envuelven al primitivo y permiten `null`. Muy usados arriba: `Long productId`, `Integer quantity`.
- **BigDecimal**: aritmética decimal exacta. Ideal para dinero porque `double` tiene errores de representación ($0.1 + 0.2 = 0.30000000000000004$).

Pregunta típica: '0.1 + 0.2 en Java da 0.3?' -> NO con double; con BigDecimal y compareTo() SI. Por eso en Cash se compara con `compareTo` y no con `==`.

1.4 Cadenas, fechas y null-safety

El proyecto usa `java.time` (API moderna desde Java 8) para fechas: `LocalDate` (dia/mes/año) y `LocalDateTime` (con hora). Ejemplo real en el bus de ventas:

- `String`: inmutable y con pool. Se comparan con `equals()`, nunca con `==`.
- `Optional`: contenedor que expresa 'puede ser null'. Evita NPE y fuerza a manejar la ausencia. Clasico en repositorios:

En entrevista: '`Optional` no se usa como parametro ni como campo, es para retornos de metodos que pueden estar vacios'. Aqui, `findByEmail` devuelve `Optional` y la logica de login encadena `filter/map/orElse`: programacion funcional real.

2. Fundamentos del lenguaje aplicados al proyecto

2.1 Modificadores de acceso y encapsulacion

Las entidades del proyecto exponen sus campos via `getters/setters` (encapsulacion) y los atributos son `private`:

Modificador	Alcance	En el proyecto
<code>public</code>	Cualquier clase de cualquier paquete	Metodos de controllers y services
<code>private</code>	Solo la misma clase	Campos de entidades; metodos helper (<code>validateRequest</code> , <code>isJpeg</code> ...)
<code>protected</code>	Mismo paquete + subclases	No usado en entidades
(paquete)	Solo el mismo paquete	Default: algunas clases helper

2.2 static vs instancia y constantes

- `static` pertenece a la clase, no al objeto: `ReportsMapper.toPaymentMethodDTO(...)` se invoca sin instanciar. Las constantes son `static final` (mayusculas con guion bajo).
- `switch expression` (Java 14+) devuelve un valor con `->`; el viejo `switch` era solo `statement`. Tambien se usa en `ReportsImpl` y maneja `yield` cuando hay varios `statements`.

2.3 Streams y expresiones lambda

Un `Stream` es una secuencia de elementos sobre la que se encadenan operaciones: `intermedias` (`map`, `filter`, `sorted`) que son perezosas, y `terminales` (`collect`, `toList`, `forEach`) que disparan el calculo. Este patron esta por todo el backend:

- `Collector`: es la terminal mas usada; tambien existen `toList()` (Java 16+) y `toMap()`.
- `lambda`: funcion anonima compacta (parametros) -> cuerpo. Ej: `p -> p.getName()`.
- `method reference`: `::`, atajo. `Comparator.comparing(MarginDTO::getMarginPercent)` equivale a `dto -> dto.getMarginPercent()`.

- `map` transforma cada elemento; `filter` lo condiciona; `sorted` ordena; `collect/toList` recoge; `findFirst` devuelve Optional.

Explica que los streams son `lazy` y `no mutan` la coleccion fuente: se lee el stream, se aplica la cadena y se produce una nueva estructura. Sirve para explicar porque las consultas de reportes agregan en SQL y solo transforman aqui.

2.4 Records, enums y text blocks

- `record`: clases inmutables brevísimas (equals/hashCode/toString automáticos). `Range(from, to)` ya tiene getters `from()` y `to()` y constructor posicional.
- `enum`: tipo cerrado con valores fijos. Aqui modelan permisos, metodos de pago (CASH/DEBIT/CREDIT), estados de pago (PENDING/APPROVED/REJECTED/REVERSED) y agrupacion de reportes (DAY/MONTH/YEAR).
- `text block`: cadenas multilinea con triple comilla. Permiten leer las queries JPQL como SQL real.

2.5 Colecciones: List, Set y Map

El proyecto usa `List` (orden, permite duplicados), `Set` (sin duplicados, para permisos del rol) y `Map` (clave->valor, no muy usado):

- `ArrayList` (linear por indice), `HashSet` (hash por equals/hashCode), `HashMap` (par clave-valor).
- `Set` fue clave para `evitar N+1` en el borrado logico de productos: se calcula una vez el Set de ids con historico y en el stream se consulta con `contains`.

2.6 Interfaces y clases abstractas

La arquitectura del proyecto separa `contrato` (interfaz) de `implementacion` (clase):

- La `interfaz` define el *qué* (abstraccion); la clase concreta el *cómo*. Usar la interfaz en los controllers permite cambiar la implementacion sin tocar al consumidor y facilita los mocks en tests.
- Ley de arquitectura del proyecto: `controller -> service (interfaz) -> service/impl -> repository -> entity`.

3. Programacion Orientada a Objetos (POO)

3.1 Los cuatro pilares, con ejemplos del proyecto

Pilar	Concepto	Ejemplo en Ventas-Backend
Encapsulacion	Ocultar estado; acceso controlado	Entidades con campos private y getters/setters
Abstraccion	Modelar conceptos relevantes, ocultar detalle	Interfaces Service (ProductService) que ocultan la logica de ProductImpl
Herencia	Reutilizar/extends; en Java simple	Excepciones extends RuntimeException; impls implementan interfaces

Pilar	Concepto	Ejemplo en Ventas-Backend
Polimorfismo	Misma interfaz, multiples comportamientos	PaymentService/TerminalService con TerminalSimulatedImpl y TerminalPhysicalImpl

3.2 Polimorfismo real: la terminal de pagos

Este es el mejor ejemplo de POO del proyecto: dos implementaciones distintas de la misma abstraccion seleccionables por configuracion:

El controller de pagos depende de la **interfaz** `TerminalService`; en tiempo de ejecucion Spring inyecta el bean correcto segun la propiedad `payment.terminal.type`. Eso es polimorfismo + inversion de dependencias.

3.3 Composicion sobre herencia

El proyecto prefiere **composicion**: inyecta dependencias en el constructor (has-a) en vez de heredar comportamiento (is-a). Cada service 'compone' repositories y otros services: `SaleImpl` tiene `ProductRepository`, `SaleRepository`, `CashRegisterRepository` y `PaymentService` y `SaleMapper`.

4. Maven y Spring Initializr

4.1 Que es Spring Initializr

Spring Initializr (start.spring.io) es la web que genera el esqueleto del proyecto: elige lenguaje (Java), version de Spring Boot, grupo/artefacto y dependencias, y descarga un zip con el `pom.xml` o `build.gradle` listo. Para este proyecto se eligio: **Spring Boot 4.0.5, Java 21, Maven**, con los starters: data-jpa, web, security, validation, mail, h2 (tests), postgresql y jwt (JWT).

4.2 Anatomia del pom.xml

- **spring-boot-starter-parent**: gestiona versiones de todas las librerias (no hay que indicar versiones de starters).
- **Starters**: dependencias descriptoras que traen todo lo de una funcionalidad (web incluye Tomcat + Jackson + Spring MVC).
- **Lombok**: reduce boilerplate con anotaciones; aqui se usa de forma parcial (p. ej. `@Slf4j` para logs y `@Data` en `TerminalConfig`).
- **scope runtime** en el driver de BD: solo se necesita al ejecutar, no al compilar.

4.3 Maven Wrapper y ciclos de vida

El proyecto tiene `mvnw.cmd`: el **Maven Wrapper**, que descarga la version de Maven indicada y permite ejecutar sin tenerlo instalado. Fases tipicas:

Comando	Que hace
mvnw.cmd compile	Compila src/main/java a target/classes
mvnw.cmd test	Compila y ejecuta los tests de src/test
mvnw.cmd package -DskipTests	Genera el jar (lo usa el Dockerfile)
mvnw.cmd spring-boot:run	Compila y arranca la app embebida

5. Spring Core: Inversion de Control (IoC) y DI

5.1 Que es el contenedor de Spring

Spring gira en torno a un **contenedor IoC** que crea y administra los **beans** (objetos). En vez de que cada clase haga `new` de sus dependencias, **Spring las inyecta** (Inyeccion de Dependencias). El patrón es: **no me busques tu, yo te paso lo que necesitas**.

5.2 Anotaciones de estereotipo (como se registran los beans)

En `VentasApplication`, `@SpringBootApplication` incluye `@ComponentScan` (y auto-configuracion y `@Configuration`). Escanea el paquete `com.erikjarquin.ventas` y subpaquetes (config, controller, service, repository, model, mapper, exceptions, jobs).

5.3 Inyeccion por constructor (la recomendada)

- Desde Spring 4.3, si la clase tiene UN solo constructor ya no hace falta `@Autowired`: **constructor injection**.
- Por que constructor sobre campo: beans **inmutables** (final), testabilidad (inyectas mocks facil), sin *half-initialized* objects y deteccion temprana de dependencias circulares.

En entrevista hay 3 formas: 1) campo con `@Autowired` (NO recomendado), 2) setter con `@Autowired` (dependencia opcional), 3) constructor (recomendada). Pon ejemplos del proyecto: todos los beans usan constructor.

5.4 @Value y @ConfigurationProperties

- `@Value` para una propiedad suelta; `@ConfigurationProperties` para datos estructurados y tipados.
- El prefijo `payment.terminal` se mapea desde `application.yml`: `type: ${PAYMENT_TERMINAL_TYPE:SIMULATED}`.

6. Spring Boot: auto-configuracion, starters y profiles

6.1 @SpringBootApplication

Es metaanotacion = `@SpringBootApplication` + `@EnableAutoConfiguration` + `@ComponentScan`. La **auto-configuracion** confecciona beans por convencion segun lo que hay en el classpath (si esta H2, configura H2; si esta Spring Security, activa la cadena de filtros, etc.).

6.2 application.yaml + application-local.yaml + variables de entorno

Este es un punto fuerte para la entrevista porque el proyecto aplica la **externalizacion de configuracion** de Spring:

- **Placeholders**: `${VAR}` o `${VAR:default}`.
- **Precedencia** (de mayor a menor): variables de entorno del sistema / Railway > application-local.yaml (perfil 'local') > application.yaml.
- **application-local.yaml** esta en .gitignore: contiene los valores reales de dev (DB_USER, DB_PASSWORD, JWT_SECRET, MAIL_PASSWORD, ADMIN_EMAIL...). En Railway no existe: los valores se ponen en las Variables del servicio.
- Si una `${VAR}` sin default no se resuelve, la app **no arranca** (fail-fast), a proposito: evita correr con valores vacios.

Spring Profiles: `spring.profiles.default: local` activa application-local.yaml en dev. En prod podrias tener application-prod.yaml y lanzar con `SPRING_PROFILES_ACTIVE=prod`.

6.3 Servidor embebido y multipart

Spring Boot empaqueta un **Tomcat embebido**: no hay que instalar servidor externo. `server.port=${PORT:8081}` hace que en Railway (que inyecta \$PORT) use el puerto asignado, y en local el 8081. El multipart (20MB) se configuro para subir imagenes de productos.

7. Spring MVC: construccion de APIs REST

7.1 @RestController y mapeo de rutas

Anotacion	Metodo HTTP	Uso en el proyecto
@GetMapping	GET	Listados, detalle, reportes, search
@PostMapping	POST	Crear: login, productos, ventas, compras, pagos
@PutMapping	PUT	Editar integral: /api/products/{id}, /api/categories/{id}
@PatchMapping	PATCH	Cambio parcial: /{id}/active (activar), /{id}/deactivate
@DeleteMapping	DELETE	Borrado: /{id} (productos, compras cancel, roles)

7.2 Params, body y response

- `@PathVariable`: valor de la URL -> /status/{transactionId} -> `String transactionId`.

- `@RequestParam`: parametro query -> `?q=cafe`, `?category=Bebidas`, `?limit=5`; con `required=false` o `defaultValue`.
- `@RequestBody`: JSON del body -> se deserializa con Jackson al DTO.
- `ResponseEntity`: permite controlar el `status HTTP` y el body con flexibilidad:

Convenciones REST que puedes citar: POST para crear (201 CREATED), GET para leer (200), DELETE para borrar (204 No Content), PUT completo / PATCH parcial, errores con 4xx/5xx y cuerpo de error JSON uniforme.

7.3 Serializacion JSON y @JsonIgnore

Jackson (incluido en web) convierte el objeto de retorno a JSON. Para evitar `ciclos` en relaciones bidireccionales se usa `@JsonIgnore`:

8. Spring Data JPA: entidades, repositorios y JPQL

8.1 Anotaciones de mapeo objeto-relacional

Anotacion	Tipo de relacion	Ejemplo real	Fetch
@ManyToOne	N -> 1 (padre leve)	UserEntity.role	LAZY
@OneToMany	1 -> N (coleccion)	SaleEntity.details, CategoryEntity.products	LAZY
@OneToOne	1 -> 1	SaleEntity.payment (mappedBy)	LAZY
@ManyToMany	N <-> N (tabla puente)	RoleEntity.permissions (role_permissions)	EAGER

8.2 FetchType LAZY vs EAGER y el problema de rendimiento

Es tema clavado de entrevista y este proyecto lo muestra en ambos lados:

- **EAGER** (ansioso): carga la relacion SIEMPRE de inmediato. En `RoleEntity.permissions` se usa EAGER porque `SecurityAuthorityMapper` necesita los permisos en cada request autenticado y asi evita `LazyInitializationException`.
- **LAZY**: carga la relacion solo cuando se accede. Evita traer todo en cada consulta (rendimiento/performance). En `UserEntity.role` y `ProductEntity.category` se usa LAZY para evitar consultas dobles innecesarias.

Peligro del LAZY + fuera de transaccion: `LazyInitializationException`. El proyecto lo evita con: 1) JOIN FETCH en repositorios, 2) `@EntityGraph`, 3) conteos SQL (`countByRole_Id`) en lugar de tocar colecciones lazy cuando se borra un rol/categoria. Menciona 'open-in-view' como anti-patron que Spring Boot activa por defecto.

8.3 JpaRepository: metodos derivados

Spring Data **parsea** el nombre del metodo y deriva la query: `findBy` + propiedad + operador. Es la magia de `JpaRepository`: metodos de dominio sin escribir SQL.

8.4 JPQL: @Query, :parametros y JOIN FETCH

- **JPQL**: consultas sobre entidades y sus campos (`UserEntity`), no sobre tablas. Spring lo traduce a SQL para el dialecto (PostgreSQL/H2).
- **JOIN FETCH**: trae la relacion en la MISMA query SQL (genera un INNER JOIN). Solucion canonica al N+1.
- **@Param**: vincula parametros posicionales por nombre, evita inyeccion (queries parametrizadas).

8.5 Paginacion y orden

- `Pageable`/`PageRequest` agregan LIMIT/OFFSET al SQL real. Aqui limita el top de productos a 1-100 validado en service.

8.6 EntityGraph y N+1 en listados

Para listar compras sin disparar una query por cada proveedor/detalle/producto se usa `@EntityGraph`, que cumple lo mismo que JOIN FETCH pero declarando los atributos a precargar. En entrevista: **'listado con EntityGraph para evitar N+1'** suena muy senior.

9. Transacciones y el problema N+1

9.1 @Transactional

Una **transaccion** agrupa operaciones: o todas se aplican o ninguna (ACID). En este proyecto la transaccion mas importante es la **venta**: si el pago con tarjeta falla, se hace **rollback total**.

- **readOnly = true**: indica que no habra escrituras; Hibernate no hace dirty-checking de escritura.
- **rollbackFor = Exception.class**: por defecto Spring solo hace rollback con RuntimeException; con esto tambien con checked.
- El controlador declara la transaccion (*el contenedor* la abre), NO el controller. El **proxy** de Spring envuelve el bean del service.

9.2 El problema N+1 y sus soluciones

Es la pregunta de rendimiento favorita en entrevistas:

- Problema: cargas una lista de N padres y luego accedes a una relacion lazy de cada uno -> 1 query (padres) + N queries (por cada hijo).
- Ejemplo: listar 100 compras y por cada una acceder a sus detalles -> 1 + 100 queries.
- Soluciones aplicadas en el proyecto: **@EntityGraph** (`PurchaseRepository`), **JOIN FETCH** (`UserRepository.findByEmailWithRoleAndPermissions`), **calculos** en dos consultas con **Set** (`ProductImpl.productIdsWithHistory`) y **conteos SQL** (`countByRole_Id`) para no tocar LAZY fuera de transaccion.

10. Spring Security: JWT, filtros y autorizacion

10.1 SecurityFilterChain

- **CSRF desactivado**: sin sesiones con cookies, el token viaja en header, no hay ataque de csrf clasico.
- **CORS**: controla que origenes del navegador pueden llamar ({CORS_ALLOWED_ORIGINS}, separado por comas).
- **@EnableMethodSecurity**: activa `@PreAuthorize` a nivel de metodo.

10.2 Autorizacion por PERMISO con @PreAuthorize

Los **31 permisos** (34 segun AGENTS; hoy 32 tras quitar VER_CLIENTES/VER_FACTURAS) viven en el enum `PermissionName` y se asocian a roles. El string de `hasAuthority` debe coincidir EXACTO con el nombre del enum.

10.3 JwtFilter + JwtUtil: flujo de autenticacion

- **OncePerRequestFilter**: garantiza ejecucion una sola vez por request (aunque haya otros filtros).
- **JWT solo lleva el email** (subject). Los permisos se recargan de la BD en CADA request: si quitas un permiso, aplica de inmediato sin esperar expiracion del token.
- El **SecurityContextHolder** guarda la Authentication; de ahi la agarran los `@PreAuthorize` y el controller via `Authentication auth` en `AuthController.changePassword`.

10.4 JwtUtil: firma HS256 con jjwt

- HS256 = HMAC-SHA256 con **secreto simetrico** (el mismo se firma y verifica).
- El **JWT_SECRET** viene de variable de entorno nunca hardcodeado: en local en application-local.yaml (gitignored), en Railway en las Variables. Si se filtra, cualquiera puede forjar tokens de admin.
- Expiracion: **5 horas**. Los claim (email) -> subject; issuer/audience podrian agregarse (nivel senior).

10.5 Roles vs Permisos: SecurityAuthorityMapper

- **hasRole('ADMIN')** busca `ROLE_ADMIN`;
hasAuthority('VER_PRODUCTOS') busca exacto. El proyecto usa el segundo por permisos granulares (RBAC: role-based access control).

10.6 BCrypt y el login

BCrypt es un hash con **salt** y work factor: guardar contraseñas en texto plano es fatal. El login devuelve `LoginResponse(success=false)` tanto si el email no existe como si la password falla: se evita **enumeracion de usuarios** (buena respuesta de senior).

11. Manejo global de excepciones

11.1 @RestControllerAdvice + @ExceptionHandler

En vez de try/catch en cada controller, se centraliza la conversion de errores en un consejo global. Cualquier excepcion de negocio se transforma en una respuesta JSON uniforme:

11.2 Excepciones de negocio con HttpStatus

- Del lado frond-end (Angular) se lee la respuesta `<font face='Courier'>{timestamp, status, code, message}</font>` y se muestra el error al usuario (p. ej. 409 en Roles/Categorias).
- Otros: `<font face='Courier'>UserException</font>`, `<font face='Courier'>RoleException</font>`, `<font face='Courier'>CategoryException</font>`, `<font face='Courier'>ProviderException</font>`, `<font face='Courier'>PurchaseException</font>`, `<font face='Courier'>CashException</font>`, `<font face='Courier'>PaymentException</font>`, `<font face='Courier'>SaleException</font>`.

12. DTOs y Mappers

12.1 Por que no exponer las entidades

Responder esta pregunta en entrevista te destaca: **seguridad** (no filtras password/resetToken), **desacoplamiento** (la entidad puede cambiar sin romper el contrato de la API) y **enriquecimiento** (URL de imagen completa, hashHistory, labels).

- Entidades: mapean la BD (JPA). DTOs: mapean lo que viaja por HTTP. Mappers: transforman.
- Nomenclatura: colocados en `<font face='Courier'>model/dto/<Modulo></font>` (Login, Products, Sale, Purchases, Reports, ResetPassword, Terminal, Cash, Role, Categories, User, Permissions, Payment).

12.2 Tipos de mappers en el proyecto

La pregunta de cuando convertir un mapper en bean: cuando necesita configuracion/dependencias. `<font face='Courier'>ProductMapper</font>` fue `@Component` justamente para inyectar `<font face='Courier'>app.upload-url</font>` y devolver URL completa (el frontend Angular usa `<font face='Courier'>[src]="p.img"</font>` directamente).

13. Testing: JUnit 5, Mockito, MockMvc

13.1 Piramide de testing del proyecto

Tipo de test	Anotacion	Alcance	Ejemplos
Unitario de servicio	@ExtendWith(MockitoExtension.class)	Service con repos mockeados	ProductImplTest, PurchaseImplTest
De repositorio	@DataJpaTest + H2	JPQL y relaciones contra BD real en memoria	UserRepositoryTest, ProductRepositoryTest, ReportQueryTest, ProviderRepositoryTest
De controlador	@WebMvcTest + MockMvc + @WithMockUser	Mapeo HTTP + seguridad sin levantar Servidor	ProductControllerTest, AuthControllerTest...
De config	Mockito plain	Bootstraps	RolePermissionBootstrapTest
De contexto	@SpringBootTest	Arranca la app	VentasApplicationTests

13.2 Test unitario de servicio con Mockito

- `@Mock`: crea un doble del repositorio (sin BD).
- `when(...).thenReturn(...)`: define comportamiento.
- `verify(...)`: comprueba que se llamo (veces, argumentos). `never()` para lo no permitido.

13.3 Test de repositorio con H2 (@DataJpaTest)

Importante: JPQL portable (`CAST(s.saleDate AS date)`, `year()`/`month()`) corre igual en H2 (tests) y PostgreSQL (prod). Eso lo verifica `ReportQueryTest`.

13.4 Test de controlador con MockMvc y seguridad

- `MockMvc`: simula peticiones HTTP sin levantar Tomcat (velocidad).
- `@WithMockUser`: autentica sin generar JWT; `authorities` modela los permisos.
- `jsonPath`: asserts sobre el JSON de respuesta (status, campos).
- `@MockitoBean`: en Spring Boot 4 reemplaza a `@MockBean`; crea un mock que se mete en el contexto.

13.5 Test del bootstrap (configuracion)

Estos tests garantizaron la regla del flag `APP_SEED_BOOTSTRAPS`: en false, los bootstraps no tocan roles/permisos; y los roles solo se crean si la tabla esta vacia (no se 'resucitan' roles borrados a mano).

14. Temas avanzados de entrevista

14.1 ddl-auto: update vs validate (y migraciones)

`spring.jpa.hibernate.ddl-auto` controla como Hibernate toca el esquema:

Valor	Comportamiento	Para que
none	No toca nada	Prod administrada
validate	Solo valida que el esquema coincida, NO modifica	Recomendado en prod madura
update	Crea/actualiza tablas y columnas nuevas	Este proyecto en dev (<code>{JPA_DDL_AUTO:update}</code>)
create	Borra y recrea cada arranque	Nunca en prod (pierde datos)
create-drop	Crea y borra al cerrar	Tests

Limitacion que el proyecto detecto: `update` agrega columnas/tablas, pero NO modifica constraints existentes (el `CHECK permissions_name_check` generado por `@Enumerated`). Por eso hubo que droppear constraints a mano en BD local. La solucion de arquitectura seria `Flyway/Liquibase` + `validate`. Puedes decir esto en la entrevista: demuestra que conoces las limitaciones y las soluciones.

14.2 REST y HTTP (status codes bien usados)

Status	Significado	Ejemplo en el proyecto
200 OK	Exito	Listados, login ok
201 CREATED	Recurso creado	POST /api/purchases
204 No Content	Borrado ok	DELETE /api/products/{id}
400 Bad Request	Entrada invalida	Stock insuficiente, items vacios, validacion @Valid
401 Unauthorized	Sin autenticacion	Sin JWT o token invalido
403 Forbidden	Autenticado sin permiso	@PreAuthorize denegado
404 No Found	No existe	Producto/venta/rol no encontrado
409 Conflict	Violacion de regla de negocio	Producto con ventas, RFC duplicado, rol con usuarios

14.3 Seguridad real: lo que BBVA va a querer oír

- Contraseñas con `BCrypt` (salt + work factor), nunca texto plano.
- JWT en header `Authorization: Bearer`, secretos por `variables de entorno` (nunca en git; `application-local.yaml gitignored`).
- `RBAC` con permisos granulares (32+ permisos) y evaluados por BD en cada request: revocacion inmediata.
- `Login` que no filtra existencias de email (`success=false` generico).
- CSRF desactivado por ser `stateless`; CORS restringido por orígenes. Correo SMTP con `app-password` de Gmail, no la password normal.
- Subida de imagenes validada por `tamano + extension + magic bytes` (`FileStorageService`): un .exe renombrado a .jpg se rechaza. Nombre almacenado `UUID` para evitar path traversal.
- Purgar secretos del historial git con `git filter-repo` antes de publicar.

14.4 Idempotencia y reintentos de pagos

- Estados del pago: `PENDING, PROCESSING, APPROVED, REJECTED, REVERSED`.
- `PaymentMonitorJob` (`@Scheduled`, cada 5 min) resuelve pagos `PENDING` viejos consultando la terminal: recupera cobros cuyo ACK se perdio en la red.
- `reversePayment` solo aplica sobre `APPROVED` y marca pago y venta como `REVERSED`.

Idempotencia: atrapar el mismo cobro dos veces sin duplicar cobro. Menciona que el `transactionId` se genera desde el id de la venta para persistencia en reintentos.

14.5 Contexto de Docker y despliegue

- Multi-etapa: la imagen final no trae Maven ni código fuente, solo el jar + JRE.
- Usuario no-root (UID 1001): minimiza superficie de ataque.
- .dockerignore excluye application-local.yaml y .env (secretos NO entran al contenedor) y target/uploads.
- Railway: variables del servicio (DB_*, JWT_SECRET...), volumen en /app/uploads para las imágenes, puerto \$PORT.

14.6 Fail-fast y semántica de nulls

`orElseThrow` transforma 'no encontrado' en excepción controlada; `IllegalArgumentException` -> el handler global la convierte en 400. Ejemplo repetido en `ProductImpl` y `RoleImpl`. También se usa `filter` en vez de ifs apilados en `AuthImpl`.

14.7 Rendimiento y concurrencia (nivel senior)

- N+1 y sus soluciones (`EntityGraph` / `JOIN FETCH` / Set de ids).
- Fetches con `Pageable` en queries agregadas (top N).
- `Set` para deduplicar en memoria y evitar exists por producto.
- Consultas de reportes **agregan en SQL**, no en Java: el frontend solo grafica; se evita traer millones de filas al servidor.
- Concurrencia en stock: hoy restringida por transacción y validación de stock; podrías mencionar **pessimistic/optimistic locking** (`@Version`) como mejora si dos cajeros venden el último producto en paralelo.

15. Caso de estudio end-to-end: flujo de una VENTA con tarjeta

Recorre esto de memoria: es la mejor manera de mostrar que dominas TODO el stack en una sola respuesta larga.

Paso 1 - Llega el request HTTP

Paso 2 - El controller delega en el service

Paso 3 - La transacción en SaleImpl

- Si `processCardPayment` lanza (rechazo/timeout): `SaleImpl` marca REJECTED y **relanza**. El `@Transactional(rollbackFor=Exception.class)` hace **rollback** de todo: ni la venta PENDING ni el pago ni stock. Eso es integridad transaccional real.

Paso 4 - La respuesta

Y después? (flujos secundarios)

- **Reporte**: GET /api/reports/trend llama a SaleRepository (JPQL que filtra APPROVED y agrega por CAST(fecha)) -> ReportsMapper -> DTO -> Chart.js en Angular.
- **Compra**: POST /api/purchases suma stock y setea product.cost -> los 'margins' del dashboard (price-cost).
- **Reversa**: POST /api/payments/reverse/{id} (solo APPROVED) -> terminal REV| -> pago y venta en REVERSED; dejan de contar en reportes porque filtran por APPROVED.
- **Monitor**: si el pago quedo PENDING, PaymentMonitorJob lo resuelve consultando la terminal cada 5 min.

Anexo: checklist rapido antes de la entrevista

Tema	Frase poderosa
IoC/DI	Constructor injection, beans inmutables, interfaces service/impl para testear con mocks
Spring Boot	Auto-configuracion segun classpath, starters, profiles, externalizacion con \${VAR}
JPA	Lazy/Eager, N+1 resuelto con JOIN FETCH y @EntityGraph, queries derivadas y JPQL con @Param
Transacciones	@Transactional con rollbackFor en la venta: si el pago falla, rollback total
Seguridad	JWT HS256 firma con secret de entorno, permisos RBAC recargados de BD por request, BCrypt
Excepciones	@RestControllerAdvice + @ExceptionHandler con JSON uniforme {timestamp,status,code,message}
Testing	JUnit5 + Mockito (unitarios), @DataJpaTest con H2 (repos), @WebMvcTest + @WithMockUser (controllers)
APIs	REST: 201 CREATED, 204 No Content, 400/401/403/404/409 con significado
Dinero	Siempre BigDecimal (nunca double) y compareTo, no ==
Prod	ddl-auto update vs validate, migraciones (Flyway), secretos fuera de git, Docker multi-etapa no-root